

Foundations of Robotics

Prof. Dr. Oskar von Stryk, Jonathan Lichtenfeld

Winter Semester 2025 / 2026



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Solution Proposal for Exercise 6

Task 1: Differential-driven, three-wheeled vehicles

Differentialgetriebene, dreirädrige Fahrzeuge

(6 Points)

The following uses a differential-driven vehicle (cf. lecture, chapter 4.5.2).

Im Folgenden wird ein differentialgetriebenes Fahrzeug verwendet (vgl. Vorlesung Kapitel 4.5.2).

a) Solve the forward kinematics problem for the special case that the wheel velocities are constant and different: $v_l(t) = v_l = \text{const}$ and $v_r(t) = v_r = \text{const}$ with $v_l \neq v_r$, i.e. compute $x(t)$, $y(t)$, $\theta(t)$.

Lösen Sie das Vorwärtskinematikproblem für den Sonderfall, dass die Geschwindigkeiten der Räder konstant und unterschiedlich sind: $v_l(t) = v_l = \text{const}$ und $v_r(t) = v_r = \text{const}$ mit $v_l \neq v_r$, d. h. berechnen Sie $x(t)$, $y(t)$, $\theta(t)$.

- What trajectories of the vehicle result in the world coordinate system?

Welche Bahnen des Fahrzeugs ergeben sich im Weltkoordinatensystem?

- Can all points of a plane be reached by these trajectories?

Lassen sich mit diesen Bahnen alle Punkte einer Ebene erreichen?

Solution:

From the lecture the following equations are known:

$$\begin{aligned} \dot{x}(t) &= V(t) \cdot \cos \theta(t), & x(0) &= x_0, \\ \dot{y}(t) &= V(t) \cdot \sin \theta(t), & y(0) &= y_0, \\ \dot{\theta}(t) &= \omega(t), & \theta(0) &= \theta_0 \end{aligned} \quad \Rightarrow \quad \begin{cases} x(t) = x_0 + \int_0^t V(\tau) \cdot \cos \theta(\tau) d\tau, \\ y(t) = y_0 + \int_0^t V(\tau) \cdot \sin \theta(\tau) d\tau, \\ \theta(t) = \theta_0 + \int_0^t \omega(\tau) d\tau. \end{cases}$$

$$V(t) = \frac{1}{2}(v_l(t) + v_r(t)), \quad \omega(t) = \frac{1}{b}(v_r(t) - v_l(t)), \quad R(t) = \frac{b}{2} \frac{v_r + v_l}{v_r - v_l} = \frac{V}{\omega}$$

Given:

$$v_l(t) = \text{const} = v_l$$

$$v_r(t) = \text{const} = v_r$$

From this follows:

$$\begin{aligned}\omega(t) &= \omega = \frac{1}{b}(v_r - v_l) \\ V(t) &= V = \frac{v_l + v_r}{2} \\ R(t) &= R = \frac{V}{\omega} = \frac{b}{2} \cdot \frac{v_r + v_l}{v_r - v_l}\end{aligned}$$

Thus the vehicle pose is

$$\begin{aligned}\theta(t) &= \theta_0 + \int_0^t \omega(\tau) d\tau = \theta_0 + \int_0^t \omega d\tau = \theta_0 + \omega \int_0^t d\tau \\ &= \theta_0 + \omega(t - 0) = \theta_0 + \omega \cdot t\end{aligned}\tag{1}$$

$$\begin{aligned}x(t) &= x_0 + \int_0^t V(\tau) \cos(\theta_0 + \omega\tau) d\tau \\ &= x_0 + V \left[\frac{1}{\omega} \sin(\theta_0 + \omega\tau) \right]_0^t = x_0 + \frac{V}{\omega} (\sin(\theta_0 + \omega t) - \sin(\theta_0)) \\ &= x_0 + R (\sin(\theta_0 + \omega t) - \sin(\theta_0))\end{aligned}\tag{2}$$

$$\begin{aligned}y(t) &= y_0 + \int_0^t V(\tau) \sin(\theta_0 + \omega\tau) d\tau \\ &= y_0 + V \left[-\frac{1}{\omega} \cos(\theta_0 + \omega\tau) \right]_0^t = y_0 + \frac{V}{\omega} (-\cos(\theta_0 + \omega t) + \cos(\theta_0)) \\ &= y_0 + R (\cos(\theta_0) - \cos(\theta_0 + \omega t))\end{aligned}\tag{3}$$

$$\Rightarrow \begin{pmatrix} x(t) \\ y(t) \\ \theta(t) \end{pmatrix} = \begin{pmatrix} x_0 - R \sin(\theta_0) \\ y_0 + R \cos(\theta_0) \\ \theta_0 \end{pmatrix} + \begin{pmatrix} R \cdot \sin(\theta_0 + \omega t) \\ -R \cdot \cos(\theta_0 + \omega t) \\ \omega t \end{pmatrix}\tag{4}$$

(1.5 points, 0.5 points per correct component of the RAN)

The vehicle moves on a circular trajectory with radius $R = \frac{b}{2} \frac{v_r + v_l}{v_r - v_l}$ around the ICC at $(x_0 - R \sin(\theta_0), y_0 + R \cos(\theta_0))$. It starts at (x_0, y_0) with initial orientation θ_0 . *(0.5 points)*

The centers of all reachable circles lie on the line orthogonal to the initial heading, therefore the initial heading is tangent to all such circles. Since $v_l \neq v_r \Rightarrow r < \infty$, points along the initial heading line (except the start) are not reachable. *(0.5 points)*

b) In this task a vehicle shall be driven to the goal point $(x_g, y_g) = (4, 8)$ ($g \Rightarrow \text{Goal}$), see Fig. 1. The wheelbase is $b = 2$ and the desired linear speed is $V = 1.0$. The vehicle starts at $(x_0, y_0, \theta_0) = (0, 0, 0)$. Compute constant wheel speeds v_l and v_r so that the resulting trajectory passes the goal point. At what time t_g is the goal reached? What is the vehicle orientation θ_g at the goal?

In dieser Aufgabe soll ein Fahrzeug an den Zielpunkt $(x_g, y_g) = (4, 8)$ ($g \Rightarrow \text{Goal}$) gesteuert werden (siehe Abbildung 1). Der Radabstand ist gegeben mit $b = 2$ und die gewünschte lineare Geschwindigkeit mit $V = 1.0$. Das Fahrzeug befindet sich am Ausgangspunkt $(x_0, y_0, \theta_0) = (0, 0, 0)$. Berechnen sie $v_l(t) = v_l = \text{const}$ und $v_r(t) = v_r = \text{const}$ so, dass die resultierende Bahn den Zielpunkt (x_g, y_g) schneidet. Zu welchem Zeitpunkt t_g wird das Ziel erreicht? Welche Orientierung θ_g hat das Fahrzeug am Zielpunkt?

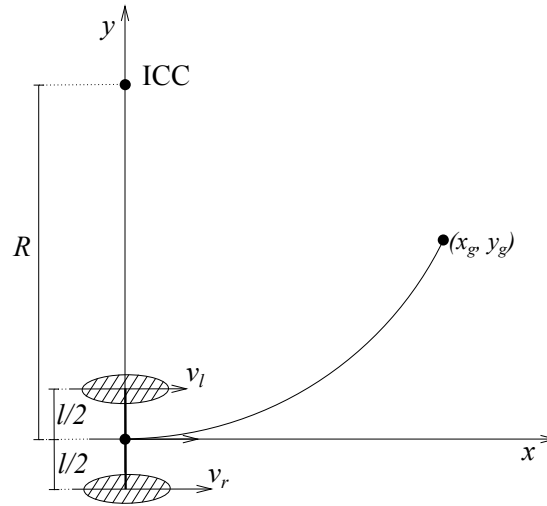


Figure 1: Differential-driven vehicle
Differentialgetriebenes Fahrzeug

Hint: Use the results from part a. First compute the trajectory radius R geometrically and then solve the linear system.

Hinweis: Verwenden Sie die Ergebnisse aus Teilaufgabe a. Bestimmen Sie zunächst den Bahnradius R geometrisch und lösen Sie anschließend das lineare Gleichungssystem.

Solution:

The vehicle trajectory is a circle. The general circle equation with center (x_m, y_m) and radius r is

$$(x - x_m)^2 + (y - y_m)^2 = r^2 \quad (5)$$

This must hold for the start and goal:

$$(x_0 - x_m)^2 + (y_0 - y_m)^2 = (x_g - x_m)^2 + (y_g - y_m)^2 = r^2 \quad (6)$$

Since the ICC lies on the line orthogonal to the start heading, we know that it is on the y-axis. This gives $x_m = 0, y_m = R$ and $r = R$.

Using this in (6) gives

$$y_m^2 = 16 + 64 - 16y_m + y_m^2 \quad (7)$$

$$\Leftrightarrow 0 = 80 - 16y_m \quad (8)$$

$$\Leftrightarrow y_m = 5 \quad (9)$$

Hence $R = 5$. (1 point)

The relation between v_l and v_r is

$$V = \frac{v_l + v_r}{2} \quad (10)$$

$$\Leftrightarrow v_r = 2V - v_l \quad (11)$$

Use V , R and ω :

$$\frac{V}{R} = \omega = \frac{v_r - v_l}{b} \quad (12)$$

$$\stackrel{(11)}{\Rightarrow} = \frac{2V - 2v_l}{b} \quad (13)$$

$$= \frac{2}{b} (V - v_l) \quad (14)$$

$$\Leftrightarrow \frac{bV}{2R} = V - v_l \quad (15)$$

$$\Leftrightarrow v_l = V - \frac{bV}{2R} \quad (16)$$

Substituting V and R in (16) yields $v_l = 0.8$. Using (11) gives $v_r = 1.2$.

(1 point, 0.5 points each for v_l , v_r)

To find t_g use the forward kinematics from part a. The circle intersects the $x = 0$ line twice between (x_0, y_0) and (x_g, y_g) ; To avoid this ambiguity, we consider the y -component from (4) (θ also possible):

$$y_g = y_0 + R \cos(\theta_0) - R \cos(\theta_0 + \omega t_g) \quad (17)$$

With the initial values:

$$y_g = R - R \cos(\omega t_g) \quad (18)$$

$$\Leftrightarrow t_g = \frac{1}{\omega} \arccos\left(\frac{R - y_g}{R}\right) \quad (19)$$

With $\omega = (v_r - v_l)/b = 0.2$ we get $t_g \approx 11.0715$. (1 point)

The orientation at the goal is obtained from (3):

$$\theta_g = \theta_0 + \omega t_g = \omega t_g \quad (20)$$

Thus $\theta_g \approx 2.2145 \approx 126.88^\circ$. (0.5 points)

Task 2: Inverse kinematics of a three-wheeled vehicle

Inverskinematik eines dreirädrigen Fahrzeuges

In this exercise the inverse kinematics of a three-wheeled differential-driven vehicle is considered. For a given target pose (x_g, y_g, θ_g) the control commands $v_r(t) = v_r = \text{const}$ and $v_l(t) = v_l = \text{const}$ shall be determined.

In dieser Aufgabe wird die Inverskinematik eines dreirädrigen differentialgetriebenen Fahrzeugs betrachtet. Hierzu sollen für eine gegebene Ziel-RAN (x_g, y_g, θ_g) die Fahrbefehle $v_r(t) = v_r = \text{const}$ und $v_l(t) = v_l = \text{const}$ ermittelt werden.

- a) Why is it not possible to provide a unique (closed-form) inverse kinematics solution to determine v_r and v_l for arbitrary target poses (x_g, y_g, θ_g) with a given time t for the differential-driven vehicle?

Warum lässt sich keine eindeutige (geschlossene) inverse Kinematiklösung zur Bestimmung von v_r und v_l für beliebige Ziel-RAN (x_g, y_g, θ_g) mit gegebenen t für das differentialgetriebene Fahrzeug angeben?

Solution:

A three-wheeled differential-driven vehicle is a non-holonomic system because it cannot move freely in all directions (e.g., sideways motion is not possible). Therefore not all target poses (x_g, y_g, θ_g) are reachable with a single constant control command. To solve this problem numerical methods or path planners are required, which combine sequences of motion primitives to reach the target pose.

- b) Compute the linear wheel velocities v_r and v_l from the controllable quantities $V(t)$ and $\omega(t)$ and, given the wheel radius r_{lr} , compute the wheel angular velocities $\dot{q}_l$ and $\dot{q}_r$.

Berechnen Sie aus den steuerbaren Größen $V(t)$ und $\omega(t)$ die linearen Radgeschwindigkeiten v_r und v_l und entsprechend dem Radradius r_{lr} die Drehgeschwindigkeit der Räder $\dot{q}_l$ und $\dot{q}_r$.

Solution:

From the lecture the following equations are known

$$\begin{aligned}v_l(t) &= \omega(t) \left(R(t) - \frac{b}{2} \right), \\v_r(t) &= \omega(t) \left(R(t) + \frac{b}{2} \right)\end{aligned}$$

With the also known relation $V(t) = \omega(t)R(t)$ one obtains the linear wheel velocities in terms of the control inputs:

$$\begin{aligned}v_l(t) &= V(t) - \omega(t) \frac{b}{2}, \\v_r(t) &= V(t) + \omega(t) \frac{b}{2}.\end{aligned}$$

And further for the wheel angular velocities

$$\begin{aligned}\dot{q}_l(t) &= \frac{v_l(t)}{r_{lr}}, \\ \dot{q}_r(t) &= \frac{v_r(t)}{r_{lr}}\end{aligned}$$

c) For the cases below, compute the controllable quantities $V(t)$ and $\omega(t)$ for the current time t from the current pose and the target pose (with case distinctions if necessary). Velocities may change discontinuously but must not exceed the limits V_{max} and ω_{max} . The target pose is considered reached when the error between current and target position and between current and target orientation is smaller than the tolerances p_{tol} and θ_{tol} . The control inputs should be computed based on the current time t for the next timestep $t + \Delta t$.

Berechnen Sie für die nachstehenden Fälle aus der aktuellen RAN des Roboters und der Ziel-RAN die steuerbaren Größen $V(t)$ und $\omega(t)$ für den aktuellen Zeitpunkt t (wenn notwendig mit Fallunterscheidung). Die Geschwindigkeit darf sprunghaft geändert werden, jedoch die Begrenzungen V_{max} und ω_{max} nicht überschreiten. Die Ziel-RAN ist erreicht, wenn der Fehler zwischen der aktuellen und Zielposition bzw. aktuellen und Zielorientierung kleiner der Toleranz p_{tol} und θ_{tol} ist. Die Steuergrößen sollen basierend auf dem aktuellen Zeitpunkt t für den nächsten Zeitschritt $t + \Delta t$ berechnet werden.

- Compute the control inputs for a rotation on the spot until the robot is oriented towards the goal.

Berechnen Sie die steuerbaren Größen für eine Rotation auf der Stelle, bis der Roboter auf die Zielposition ausgerichtet ist.

- Once the robot is oriented towards the goal, compute the control inputs for a straight drive until the goal position is reached.

Nachdem der Roboter auf die Zielposition gerichtet ist, berechnen Sie die steuerbaren Größen für eine Geradeausfahrt bis die Zielposition erreicht ist.

Hint: You may assume the system implements the control inputs perfectly and instantaneously. Also assume no slip.

Hinweis: Sie können annehmen, dass das System die Steuergrößen perfekt und instantan umsetzt. Zudem liegt kein Schlupf vor.

Solution:

Rotation on the spot

Using the differences in x and y between the robot's current position and the goal position, the orientation towards the goal can be obtained via the function `atan2`, yielding:

$$\begin{aligned} x_d &= x_g - x(t) & y_d &= y_g - y(t) \\ \theta_a &= \text{atan2}(y_d, x_d). \end{aligned}$$

To align the robot to the desired orientation, compute the rotation angle which is the difference between desired and current orientation:

$$\begin{aligned} \theta_d &= \theta_a - \theta(t) \\ \theta_{diff} &= \theta_d \bmod 2\pi = \theta_d - \left\lfloor \frac{\theta_d}{2\pi} \right\rfloor \cdot 2\pi. \end{aligned}$$

Note: To avoid unnecessary multiple rotations the angle is wrapped to the range $\pm 2\pi$. The angle θ_{diff} in this proposal is not necessarily the minimal rotation to reach the target orientation.

Overall the control inputs are

$$V(t) = 0, \text{ since only rotation}$$

$$\omega(t) = \begin{cases} 0 & , \text{ if } |\theta_{diff}| < \theta_{tol} \\ \text{sign}(\theta_{diff}) \cdot \omega_{max} & , \text{ otherwise} \end{cases}$$

Straight drive to the goal position

Once the robot is aligned to the goal, check whether the distance d to the goal is larger than the tolerance and therefore a straight drive is necessary. The control inputs are:

$$\begin{aligned} \omega(t) &= 0 & , \text{ since only straight driving} \\ V(t) &= \begin{cases} 0 & , \text{ if } d < p_{tol} \\ v_{max} & , \text{ otherwise} \end{cases} \\ d &= \sqrt{d_x^2 + d_y^2}. \end{aligned}$$

Task 3: Ackermann steering of the Santa-mobile

Ackermann-Lenkung des Santamobils

(4 Points)

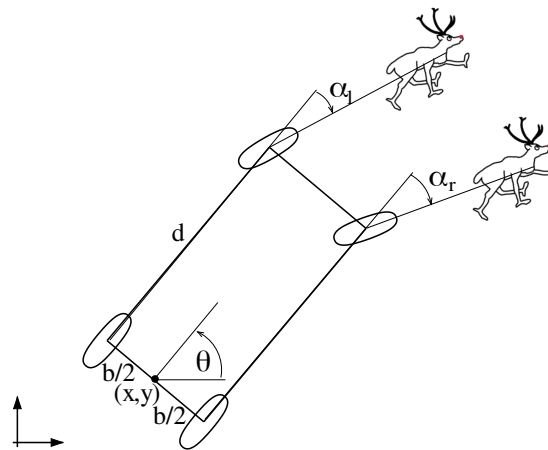


Figure 2: Four-wheeled vehicle with Ackermann steering

Since there is rarely enough snow here to use a sleigh, Santa needs a four-wheeled vehicle. Therefore this exercise deals with Ackermann steering, the steering system most commonly used in passenger cars.

Da hierzulande nur noch selten genug Schnee liegt, um einen Schlitten zum Einsatz zu bringen, benötigt der Weihnachtsmann ein vierrädriges Fahrzeug. Daher befasst sich diese Aufgabe mit der Ackermann-Lenkung, dem bei PKWs am häufigsten eingesetzten Lenksystem.

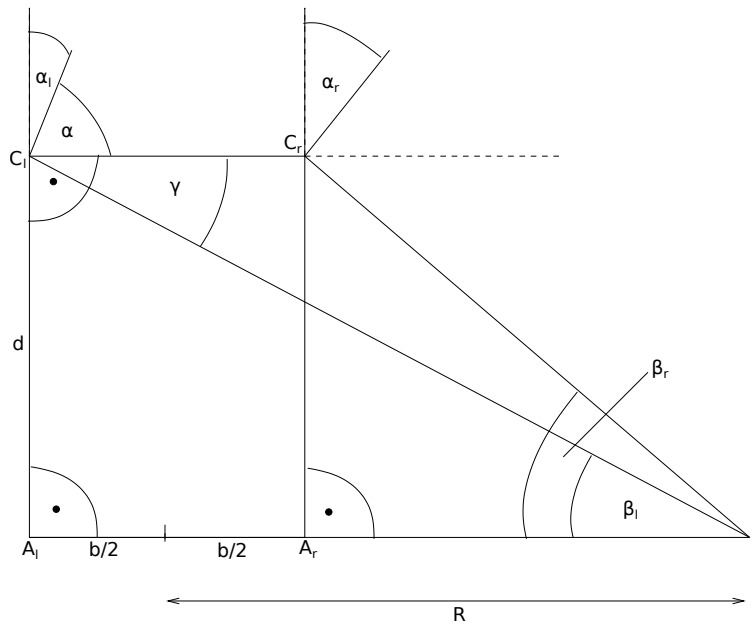
In the following, the kinematics of the motion of such a four-wheeled vehicle are examined. It has two steerable front wheels and two fixed rear wheels. The reindeer are harnessed to the front end of the vehicle and pull in the direction of the steering angles (see Figure 2).

Im Folgenden wird die zugehörige Kinematik der Bewegung eines solchen vierrädrigen Fahrzeuges untersucht. Dieses besitzt zwei lenkbare Vorderräder sowie zwei fest ausgerichtete Hinterräder. Die Rentiere sind am vorderen Ende des Gefährts angeschirrt und ziehen in Richtung der Lenkeinschlagswinkel (siehe Abbildung 2).

a) Which mathematical relations must hold between the steering angles α_r and α_l of the two front wheels and the distance R of the instantaneous center of rotation (ICC) from the vehicle reference point (x,y) for pure rolling of the vehicle? Provide the equation necessary to compute R as a function of α_r and α_l (including derivation).

Welche mathematischen Beziehungen müssen zwischen den Lenkeinschlagswinkeln α_r und α_l der beiden Vorderräder und dem Abstand R des momentanen Drehzentrums ICC vom Fahrzeugreferenzpunkt (x,y) für reines Rollen des Fahrzeugs erfüllt sein? Geben Sie die zur Berechnung von R in Abhängigkeit von α_r und α_l notwendige Gleichung an (inkl. Herleitung).

Solution:



For the left front wheel at C_l the following relations hold:

$$\alpha + \alpha_l = \frac{\pi}{2}, \quad \alpha + \gamma = \frac{\pi}{2}$$

$$\Rightarrow \alpha_l = \gamma$$

Since γ and β_l are alternate interior angles, we obtain

$$\text{for the left wheel: } \alpha_l = \beta_l,$$

$$\text{and analogously for the right wheel: } \alpha_r = \beta_r.$$

Using the tangent function the two steering angles can be computed. We consider the ratio of opposite to adjacent sides in the two triangles A_lBC_l and A_rBC_r .

The following formulas can be established:

$$\tan \alpha_l = \frac{d}{R + \frac{b}{2}} \Rightarrow R = \frac{d}{\tan \alpha_l} - \frac{b}{2} \quad (21)$$

$$\tan \alpha_r = \frac{d}{R - \frac{b}{2}} \Rightarrow R = \frac{d}{\tan \alpha_r} + \frac{b}{2} \quad (22)$$

Adding the two equations above yields the desired relation between the steering angles α_r and α_l of the two front wheels and the distance R of the instantaneous center of rotation (ICC) from the vehicle reference point (x,y) for pure rolling of the vehicle:

$$\begin{aligned}
 (21) + (22) \quad \Rightarrow 2R &= \frac{d}{\tan \alpha_l} + \frac{d}{\tan \alpha_r} = \frac{d \cos \alpha_l}{\sin \alpha_l} + \frac{d \cos \alpha_r}{\sin \alpha_r} \\
 &= \frac{d(\cos \alpha_l \sin \alpha_r + \cos \alpha_r \sin \alpha_l)}{\sin \alpha_l \sin \alpha_r} = \frac{d \sin(\alpha_l + \alpha_r)}{\sin \alpha_r \sin \alpha_l} \\
 \Leftrightarrow R &= \frac{d}{2} \cdot \frac{\sin(\alpha_l + \alpha_r)}{\sin \alpha_r \sin \alpha_l}
 \end{aligned}$$

(1 point)

b) Now consider Santa's vehicle with $b = 2\text{m}$ and $d = 5\text{m}$. The vehicle reference point shall move on a circular path with radius $R = 16\text{m}$ at a speed of $v = 4\text{m/s}$. Determine the steering angles and the rolling speeds of all four wheels for pure rolling.

Es wird nun konkret das Fahrzeug des Weihnachtsmanns mit $b = 2\text{m}$ und $d = 5\text{m}$ betrachtet. Der Referenzpunkt des Fahrzeuges soll sich auf einer Kreisbahn mit Radius $R = 16\text{m}$ mit einer Geschwindigkeit von $v = 4\text{m/s}$ bewegen. Bestimmen Sie die Einschlagwinkel und Geschwindigkeiten aller vier Räder für reines Rollen.

Solution:

The steering angles are computed using (21) and (22):

$$\alpha_l = \tan^{-1} \frac{d}{R + \frac{b}{2}} \quad \alpha_r = \tan^{-1} \frac{d}{R - \frac{b}{2}}$$

During rotation about the instantaneous center for pure rolling all points on the vehicle share the same angular velocity. Hence the linear speed of each wheel can be computed from its distance to the center of rotation and the vehicle angular velocity.

The angular velocity of the vehicle reference point is given by the linear speed of the reference point divided by the distance to the center of rotation

$$\omega = \frac{v}{R}.$$

Denoting the distances of the front wheels to the center of rotation by d_l (left wheel) and d_r (right wheel), the rolling speeds of the wheels are

$$\begin{aligned}
 v_{A_r} &= \omega \cdot \left(R - \frac{b}{2}\right) \quad v_{A_l} = \omega \cdot \left(R + \frac{b}{2}\right) \\
 v_{C_l} &= \omega \cdot d_l, \text{ with } d_l = \frac{d}{\sin(\alpha_l)} \\
 v_{C_r} &= \omega \cdot d_r, \text{ with } d_r = \frac{d}{\sin(\alpha_r)}
 \end{aligned}$$

For the concrete vehicle the computation yields:

$$\omega = \frac{4}{16} = 0.25 \text{ rad/s.}$$

Wheel	Steering angle [°]	Rolling speed [m/s]
rear left	0	4.25
rear right	0	3.75
front left	16.39	4.43
front right	18.43	3.95

(2 points for correct table, 0.25 deduction for each incorrect entry)

c) Compute the minimal turning radius of the vehicle if the front wheels can take a maximum steering angle of $\alpha_{\max}$, i.e. $0 \leq \alpha_r \leq \alpha_{\max}$ and $0 \leq \alpha_l \leq \alpha_{\max}$. The vehicle parameters are as given in b) and $\alpha_{\max} = \pi/4$.

Berechnen Sie den minimalen Drehradius des Fahrzeugs, wenn die Vorderräder einen maximalen Lenkeinschlag von $\alpha_{\max}$, also $0 \leq \alpha_r \leq \alpha_{\max}$ und $0 \leq \alpha_l \leq \alpha_{\max}$ einnehmen können. Die Parameter des Fahrzeugs sind wie in b) angegeben und $\alpha_{\max} = \pi/4$.

Solution:

The wheel that is closer to the center of rotation determines the smallest turning radius. Thus from (22) it follows

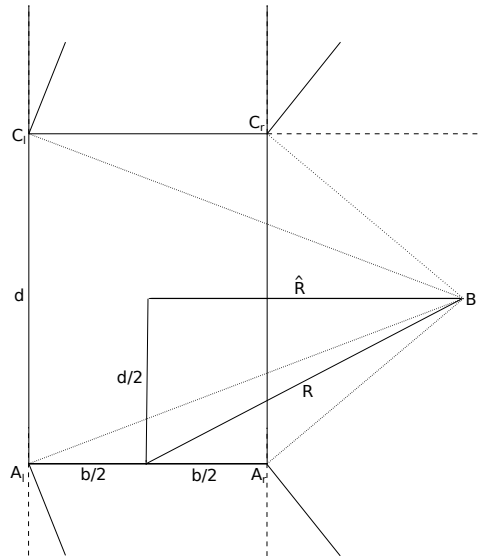
$$R = \frac{d}{\tan(\alpha_{\max})} + \frac{b}{2} = \frac{5}{\tan(\frac{\pi}{4})} + \frac{2}{2} = 6\text{m.}$$

(0.5 points)

d) How does the radius from the center of rotation to the vehicle reference point from c) change if the rear wheels are also steered by an angle ϕ analogous to the front wheels and the steering angles are $\phi_l = -\alpha_l$ and $\phi_r = -\alpha_r$?

Wie verändert sich der Radius vom Drehzentrum zum Fahrzeugreferenzpunkt aus c), wenn sich auch die Hinterräder mit einem Winkel ϕ analog zu den Vorderrädern lenken lassen und die Einschlagswinkel $\phi_l = -\alpha_l$ und $\phi_r = -\alpha_r$ sind?

Solution:



Reducing the opposite side in the consideration from a) to $\frac{d}{2}$ gives the distance from the vehicle center to the center of rotation:

$$\hat{R} = \frac{d}{2 \tan(\alpha_{max})} + \frac{b}{2} = \frac{5}{2 \tan(\frac{\pi}{4})} + \frac{2}{2} = 3.5\text{m}.$$

Using the Pythagorean theorem one obtains the desired radius from the center of rotation to the vehicle reference point:

$$R = \sqrt{\hat{R}^2 + \left(\frac{d}{2}\right)^2} = \sqrt{3.5^2 + 2.5^2} = 4.30\text{m}.$$

(0.5 points)

Programming Task P2 Differential drive of the Turtlebot3 (9.0 points)

The goal of this lab exercise is to implement the necessary functions to compute the differential drive kinematics of the Turtlebot3 and realize a basic motion controller. Implement the required parts in the provided C++ files according to the steps described below. The parameters described can be adjusted via the configuration file "diff_drive_config.yaml". For testing your solution you can use the graphical interface shown in Figure 4.

a) Forward kinematics

In the lecture you learned the equations to determine the pose of a point-like differential-driven vehicle. These are to be implemented, using the quantities measured by the robot's internal sensors, in the method `updateOdometry` (see files: `odometry.h`, `odometry.cpp`). The method receives parameters of the message types `sensor_msgs::JointState`¹ and `sensor_msgs::Imu`², which can be read via the local variables `joint_state` and `imu`. The rotational velocities of the wheels about their

¹http://docs.ros.org/api/sensor_msgs/html/msg/JointState.html

²http://docs.ros.org/api/sensor_msgs/html/msg/Imu.html

rotation axes are provided by the vector `joint_state.velocity` and can be accessed via the indices `wheel_l_idx` and `wheel_r_idx`. The time difference since the last call is available in the variable `dt`. The method shall return the member variable `odom_msg_` of type `nav_msgs::Odometry`³. Since the robot moves on the x-y plane, only the x and y position and the orientation as a quaternion (based on rotation around the z-axis) need to be updated here. The current linear and angular velocity of the robot must be updated in the fields `odom_msg_.twist.twist.linear.x` and `odom_msg_.twist.twist.angular.z`. Set all other fields to zero.

Notes:

- The angular velocity ω of the vehicle can be computed using the relations presented in the lecture or alternatively from the IMU data given as a quaternion. IMU data should, however, be smoothed (e.g. with a suitable filter) to reduce the influence of sensor noise.
- The TF library⁴ provides helper functions for handling quaternions, e.g. converting between a quaternion and the corresponding message type or computing a quaternion from yaw-pitch-roll (ROS convention: Yaw = rotation about z-axis, Pitch = rotation about y-axis, Roll = rotation about x-axis).
- You are free to add new member variables to the `Odometry` class. It is advisable to initialize them in the constructor `Odometry::Odometry(...)`.
- The following parameters relevant to you are loaded from the parameter server when the class is instantiated:

`wheel_radius_`: The wheel radii

`wheel_separation_`: Distance between the wheels

- Use the sliders at the top left of the GUI to control the robot via wheel velocities (see Figure 4).
- The odometry shown in RViz can be verified by comparison with the state in the Gazebo simulation. To do so, open the Gazebo client using the command `gzclient` in a terminal.

b) Inverse kinematics

In this subtask, given the desired angular velocity ω and linear velocity V of the vehicle, determine the corresponding wheel rotational velocities which serve as setpoints for the wheel velocity controllers. Implement the computation of the wheel rotational velocities in the method `computeWheelVelocities` of the `DiffDrive` class (see files: `diff_drive.h`, `diff_drive.cpp`). This method receives the message `twist_msg` of type `geometry_msgs::Twist`⁵, with the linear and angular velocity of the robot in `twist_msg.linear.x` and `twist_msg.angular.z`. Limit the wheel velocities to the range `[-max_wheel_vel_, max_wheel_vel_]`. Store the computed velocities in the reference parameters `wheel_l_vel` and `wheel_r_vel` passed to the function.

Notes:

- You may add new member variables to the `DiffDrive` class. It is advisable to initialize them in the constructor `DiffDrive::DiffDrive(...)`.
- The following parameters relevant to you are loaded from the parameter server when the class is instantiated:

`wheel_radius_`: Wheel radii

`wheel_separation_`: Distance between the wheels

`max_wheel_vel_`: Maximum allowed wheel velocity

- Use the controls at the bottom left of the GUI to provide linear and angular velocity commands to the robot (Figure 4).

³http://docs.ros.org/api/nav_msgs/html/msg/Odometry.html

⁴<http://docs.ros.org/api/tf/html/c++/>

⁵http://docs.ros.org/api/geometry_msgs/html/msg/Twist.html

c) Motion control

To drive the robot from the current pose (position and orientation) to a target pose, implement a motion controller. Implement this in the method `computeTwist` of the `DiffDriveControl` class (see files: `diff_drive_control.h`, `diff_drive_control.cpp`). This method receives the current robot pose `robot_pose` of type `geometry_msgs::Pose`⁶ and the time difference `time_diff` of type `ros::Duration`⁷ since the last call, and computes the control variables required to reach the goal. These are the linear velocity `twist.linear.x` and angular velocity `twist.angular.z` of the vehicle, which must be limited according to the maximum allowable velocities. The target pose is set from outside the class via the method `setGoalPose` and is then available in the member variable `goal_pose_`. First compute the difference between the current and target position and orientation, and then compute the required motion. This can, for example, be realized by executing motion primitives such as rotation on the spot and straight driving to a point. Call the method `reset()` when the target position and orientation have been reached. The method should return the variable `twist` of type `geometry_msgs::Twist`, which must be updated with the control variables you computed. The method `computeTwist` is called cyclically with the current robot pose (whenever a goal pose is available) and should compute the command for the next timestep exclusively based on the current pose.

Notes:

- To compute the control variables (linear and angular velocity), you may convert the position and orientation differences into the corresponding velocity commands by multiplying with the factor `p_gain_` (limit the resulting velocities!). The parameter `p_gain_` can be used to tune the behavior (accuracy). Be careful: setting this parameter too high may cause overshoot or unstable behavior.
- You may add new member variables to the `DiffDriveControl` class. It is advisable to initialize them in the constructor `DiffDriveControl::DiffDriveControl(...)`.
- The following parameters relevant to you are loaded from the parameter server when the class is instantiated:
 - `max_linear_vel_`, `max_angular_vel_`: Maximum allowed linear and angular velocity of the vehicle
 - `max_linear_acc_`, `max_angular_acc_`: Maximum allowed linear and angular acceleration of the vehicle
 - `pos_tolerance_`, `angular_tolerance_`: Position and orientation tolerances under which the goal is considered reached
 - `allow_backward_`: Indicates whether the vehicle is allowed to drive backwards to the goal
 - `p_gain_`: Proportional gain for converting position/orientation difference to velocities
- Acceleration limiting is **optional** and can be used to produce smoother motion.
- The GUI (Figure 4) shows a green preview of the robot. Move this preview to set a goal pose. Send the command to drive to the goal by right-clicking the preview.

Downloading and starting the programming exercise

- Before downloading the templates, first save your work. The updated or additional packages will be downloaded and built using the commands below:
 - `roscd`
 - `cd ..`
 - `git pull`
 - `turtle install tuda_exercises_student`

⁶http://docs.ros.org/api/geometry_msgs/html/msg/Pose.html

⁷http://docs.ros.org/kinetic/api/rostime/html/classros_1_1Duration.html

Teil a):
Vorwärtskinematik
(Odometrie)

Teil b):
Inverskinematik



Figure 3: After starting the simulation, the shown graphical interface appears. You can use this to test your implementation.

- turtle update
- turtle make_clean
- Start the application:
 - Start the simulation:


```
roslaunch turtlebot3_diff_drive_exercise start_sim.launch
```
 - Start the exercise:


```
roslaunch turtlebot3_diff_drive_exercise diff_drive.launch
```

Notes for the Programming Exercise

Hinweise zur Programmieraufgabe

- Please note the information provided in Moodle on the introductory slides (especially "Introduction to ROS - Part II").

Bitte beachten Sie die in Moodle bereit gestellten Informationen auf den Einführungsfolien (insbesondere "Einführung ROS - Teil II").

- The prepared virtual machine image can be downloaded via the following link: <https://moodle.informatik.tu-darmstadt.de/mod/url/view.php?id=80362>. The user's password is "ubunturos". Links to descriptions for integrating the image in VMware or VirtualBox, as well as additional notes, can be found in the slides "Introduction to Programming Exercise". For the prepared image, the installation steps from the slides "Introduction to Programming Exercise" up to slide 21 have already been executed.

Das vorbereitete Image der virtuellen Maschine kann über folgenden Link heruntergeladen werden: <https://moodle.informatik.tu-darmstadt.de/mod/url/view.php?id=80362>. Das Passwort des Benutzers lautet "ubunturos". Die Links zur Beschreibung der Einbindung des Images unter VMware oder VirtualBox sowie weitere Hinweise finden Sie auf den Folien "Einführung Programmierübung". Für das vorbereitete Image wurden die Installationsschritte aus den Folien "Einführung Programmierübung" bis Folie 21 bereits ausgeführt.

- You do not need to validate the input values of the functions.

Sie müssen die Eingabewerte der Funktionen nicht auf ihre Gültigkeit überprüfen.

- In the file `mat_nr.py` or `matNr.h`, provide the complete names and student ID numbers of all group members. You can work on the tasks in groups of up to three people. It is sufficient if one group member uploads the solution to Moodle, but the other group members must also be listed as co-authors.

Geben Sie in der Datei `mat_nr.py` bzw. `matNr.h` die vollständigen Namen und Matrikelnummern aller Gruppenmitglieder an. Sie können die Aufgaben in Gruppen von bis zu drei Personen bearbeiten. Es ist ausreichend wenn ein Gruppenmitglied die Lösung in Moodle hochlädt, jedoch müssen die weiteren Gruppenmitglieder auch als Co-Autoren dabei angegeben werden.

- To submit the programming task, compress the files to be edited according to the task description into an unencrypted `.tar.gz` or `zip` archive. Other formats are not permitted. Name the file according to the pattern `P1_1234567_7654321.zip` with the programming task number and the student ID numbers of the group members separated by underscores. Then upload this file via the file upload for the programming task in the Computer Science Learning Portal.

Für die Abgabe der Programmieraufgabe komprimieren Sie die laut Aufgabenstellung zu bearbeitenden Dateien in einem unverschlüsselten `.tar.gz`- oder `zip`-Archiv. Andere Formate sind nicht zulässig. Geben Sie der Datei den Namen nach dem Muster `P1_1234567_7654321.zip` mit der Nummer der Programmieraufgabe und den durch einen Unterstrich getrennten Matrikelnummern der Gruppenmitglieder. Laden Sie dann diese Datei im Lernportal Informatik über den zur Programmieraufgabe gehörenden Datei-Upload hoch.

- Your submitted code must be compilable for your submission to be graded.

Der von Ihnen abgegebene Code muss kompilierbar sein damit Ihre Abgabe bewertet wird.

- Up to 9 points will be awarded for the solution to the programming task.

Für die Bearbeitung der Praktikumsaufgabe werden für die Lösung bis zu 9 Punkte vergeben.

The goal of this lab exercise is to implement the necessary functions to compute the differential drive kinematics of the Turtlebot3 and realize a basic motion controller. Implement the required parts in the provided C++ files according to the steps described below. The parameters described can be adjusted via the configuration file "diff_drive_config.yaml". For testing your solution you can use the graphical interface shown in Figure 4.

a) Forward kinematics

In the lecture you learned the equations to determine the pose of a point-like differential-driven vehicle. These are to be implemented, using the quantities measured by the robot's internal sensors, in the method `updateOdometry` (see files: `odometry.h`, `odometry.cpp`). The method receives parameters of the message types `sensor_msgs::JointState`⁸ and `sensor_msgs::Imu`⁹, which can be read via the local variables `joint_state` and `imu`. The rotational velocities of the wheels about their rotation axes are provided by the vector `joint_state.velocity` and can be accessed via the indices `wheel_l_idx` and `wheel_r_idx`. The time difference since the last call is available in the variable `dt`. The method shall return the member variable `odom_msg_` of type `nav_msgs::Odometry`¹⁰. Since the robot moves on the x-y plane, only the x and y position and the orientation as a quaternion (based on rotation around the z-axis) need to be updated here. The current linear and angular velocity of the robot must be updated in the fields `odom_msg_.twist.twist.linear.x` and `odom_msg_.twist.twist.angular.z`. Set all other fields to zero.

Notes:

⁸http://docs.ros.org/api/sensor_msgs/html/msg/JointState.html

⁹http://docs.ros.org/api/sensor_msgs/html/msg/Imu.html

¹⁰http://docs.ros.org/api/nav_msgs/html/msg/Odometry.html

- The angular velocity ω of the vehicle can be computed using the relations presented in the lecture or alternatively from the IMU data given as a quaternion. IMU data should, however, be smoothed (e.g. with a suitable filter) to reduce the influence of sensor noise.
- The TF library¹¹ provides helper functions for handling quaternions, e.g. converting between a quaternion and the corresponding message type or computing a quaternion from yaw-pitch-roll (ROS convention: Yaw = rotation about z-axis, Pitch = rotation about y-axis, Roll = rotation about x-axis).
- You are free to add new member variables to the `Odometry` class. It is advisable to initialize them in the constructor `Odometry::Odometry(...)`.
- The following parameters relevant to you are loaded from the parameter server when the class is instantiated:

`wheel_radius_`: The wheel radii

`wheel_separation_`: Distance between the wheels

- Use the sliders at the top left of the GUI to control the robot via wheel velocities (see Figure 4).
- The odometry shown in RViz can be verified by comparison with the state in the Gazebo simulation. To do so, open the Gazebo client using the command `gzclient` in a terminal.

b) Inverse kinematics

In this subtask, given the desired angular velocity ω and linear velocity V of the vehicle, determine the corresponding wheel rotational velocities which serve as setpoints for the wheel velocity controllers. Implement the computation of the wheel rotational velocities in the method `computeWheelVelocities` of the `DiffDrive` class (see files: `diff_drive.h`, `diff_drive.cpp`). This method receives the message `twist_msg` of type `geometry_msgs::Twist`¹², with the linear and angular velocity of the robot in `twist_msg.linear.x` and `twist_msg.angular.z`. Limit the wheel velocities to the range `[-max_wheel_vel_, max_wheel_vel_]`. Store the computed velocities in the reference parameters `wheel_l_vel` and `wheel_r_vel` passed to the function.

Notes:

- You may add new member variables to the `DiffDrive` class. It is advisable to initialize them in the constructor `DiffDrive::DiffDrive(...)`.
- The following parameters relevant to you are loaded from the parameter server when the class is instantiated:

`wheel_radius_`: Wheel radii

`wheel_separation_`: Distance between the wheels

`max_wheel_vel_`: Maximum allowed wheel velocity

- Use the controls at the bottom left of the GUI to provide linear and angular velocity commands to the robot (Figure 4).

c) Motion control

To drive the robot from the current pose (position and orientation) to a target pose, implement a motion controller. Implement this in the method `computeTwist` of the `DiffDriveControl` class (see files: `diff_drive_control.h`, `diff_drive_control.cpp`). This method receives the current robot pose `robot_pose` of type `geometry_msgs::Pose`¹³ and the time difference `time_diff` of type `ros::Duration`¹⁴ since the last call, and computes the control variables required to reach the goal. These are the linear velocity `twist.linear.x` and angular velocity `twist.angular.z` of the vehicle, which must be limited according to the maximum allowable velocities. The target pose is set from outside the class via the method `setGoalPose` and is then available in the member variable

¹¹<http://docs.ros.org/kinetic/api/tf/html/c++/>

¹²http://docs.ros.org/api/geometry_msgs/html/msg/Twist.html

¹³http://docs.ros.org/api/geometry_msgs/html/msg/Pose.html

¹⁴http://docs.ros.org/kinetic/api/rostime/html/classros_1_1Duration.html

goal_pose_. First compute the difference between the current and target position and orientation, and then compute the required motion. This can, for example, be realized by executing motion primitives such as rotation on the spot and straight driving to a point. Call the method `reset()` when the target position and orientation have been reached. The method should return the variable `twist` of type `geometry_msgs::Twist`, which must be updated with the control variables you computed. The method `computeTwist` is called cyclically with the current robot pose (whenever a goal pose is available) and should compute the command for the next timestep exclusively based on the current pose.

Notes:

- To compute the control variables (linear and angular velocity), you may convert the position and orientation differences into the corresponding velocity commands by multiplying with the factor `p_gain_` (limit the resulting velocities!). The parameter `p_gain_` can be used to tune the behavior (accuracy). Be careful: setting this parameter too high may cause overshoot or unstable behavior.
- You may add new member variables to the `DiffDriveControl` class. It is advisable to initialize them in the constructor `DiffDriveControl::DiffDriveControl(...)`.
- The following parameters relevant to you are loaded from the parameter server when the class is instantiated:
 - `max_linear_vel_`, `max_angular_vel_`: Maximum allowed linear and angular velocity of the vehicle
 - `max_linear_acc_`, `max_angular_acc_`: Maximum allowed linear and angular acceleration of the vehicle
 - `pos_tolerance_`, `angular_tolerance_`: Position and orientation tolerances under which the goal is considered reached
 - `allow_backward_`: Indicates whether the vehicle is allowed to drive backwards to the goal
 - `p_gain_`: Proportional gain for converting position/orientation difference to velocities
- Acceleration limiting is **optional** and can be used to produce smoother motion.
- The GUI (Figure 4) shows a green preview of the robot. Move this preview to set a goal pose. Send the command to drive to the goal by right-clicking the preview.

Teil a):
Vorwärtskinematik
(Odometrie)

Teil b):
Inverskinematik



Figure 4: After starting the simulation, the shown graphical interface appears. You can use this to test your implementation.

Downloading and starting the programming exercise

- Before downloading the templates, first save your work. The updated or additional packages will be downloaded and built using the commands below:
 - `roscd`
 - `cd ..`
 - `git pull`
 - `turtle install tuda_exercises_student`
 - `turtle update`
 - `turtle make_clean`
- Start the application:
 - Start the simulation:
`roslaunch turtlebot3_diff_drive_exercise start_sim.launch`
 - Start the exercise:
`roslaunch turtlebot3_diff_drive_exercise diff_drive.launch`

Notes for the Programming Exercise

Hinweise zur Programmieraufgabe

- Please note the information provided in Moodle on the introductory slides (especially "Introduction to ROS - Part II").

Bitte beachten Sie die in Moodle bereit gestellten Informationen auf den Einführungsfolien (insbesondere "Einführung ROS - Teil II").

- The prepared virtual machine image can be downloaded via the following link: <https://moodle.informatik.tu-darmstadt.de/mod/url/view.php?id=80362>. The user's password is "ubunturos". Links to descriptions for integrating the image in VMware or VirtualBox, as well as additional notes, can be found in the slides "Introduction to Programming Exercise". For the prepared image, the installation steps from the slides "Introduction to Programming Exercise" up to slide 21 have already been executed.

Das vorbereitete Image der virtuellen Maschine kann über folgenden Link heruntergeladen werden: <https://moodle.informatik.tu-darmstadt.de/mod/url/view.php?id=80362>. Das Passwort des Benutzers lautet "ubunturos". Die Links zur Beschreibung der Einbindung des Images unter VMware oder VirtualBox sowie weitere Hinweise finden Sie auf den Folien "Einführung Programmierübung". Für das vorbereitete Image wurden die Installationsschritte aus den Folien "Einführung Programmierübung" bis Folie 21 bereits ausgeführt.

- You do not need to validate the input values of the functions.

Sie müssen die Eingabewerte der Funktionen nicht auf ihre Gültigkeit überprüfen.

- In the file `mat_nr.py` or `matNr.h`, provide the complete names and student ID numbers of all group members. You can work on the tasks in groups of up to three people. It is sufficient if one group member uploads the solution to Moodle, but the other group members must also be listed as co-authors.

Geben Sie in der Datei `mat_nr.py` bzw. `matNr.h` die vollständigen Namen und Matrikelnummern aller Gruppenmitglieder an. Sie können die Aufgaben in Gruppen von bis zu drei Personen bearbeiten. Es ist ausreichend wenn ein Gruppenmitglied die Lösung in Moodle hochlädt, jedoch müssen die weiteren Gruppenmitglieder auch als Co-Autoren dabei angegeben werden.

- To submit the programming task, compress the files to be edited according to the task description into an unencrypted `.tar.gz` or `zip` archive. Other formats are not permitted. Name the file according to the pattern `P1_1234567_7654321.zip` with the programming task number and the student ID numbers of the group members separated by underscores. Then upload this file via the file upload for the programming task in the Computer Science Learning Portal.

Für die Abgabe der Programmieraufgabe komprimieren Sie die laut Aufgabenstellung zu bearbeitenden Dateien in einem unverschlüsselten `.tar.gz`- oder `zip`-Archiv. Andere Formate sind nicht zulässig. Geben Sie der Datei den Namen nach dem Muster `P1_1234567_7654321.zip` mit der Nummer der Programmieraufgabe und den durch einen Unterstrich getrennten Matrikelnummern der Gruppenmitglieder. Laden Sie dann diese Datei im Lernportal Informatik über den zur Programmieraufgabe gehörenden Datei-Upload hoch.

- Your submitted code must be compilable for your submission to be graded.

Der von Ihnen abgegebene Code muss kompilierbar sein damit Ihre Abgabe bewertet wird.

- Up to 9 points will be awarded for the solution to the programming task.

Für die Bearbeitung der Praktikumsaufgabe werden für die Lösung bis zu 9 Punkte vergeben.

Solution Proposal